

HPC and Big Data Report

Ayomikun Abdulwahab Habeeb

Student ID: 25877639

Declaration of Originality

I declare that this assignment is all my own work and that I have acknowledged all the materials used.

Signed:

Ayomikun Abdulwahab Habeeb

HPC

1. OpenMP Performance Analysis and Optimisation

The program processes a dataset consisting of a vector and a matrix, computing statistical values such as mean, variance, minimum and maximum absolute values, and the dominant eigenvalue. The main focus is to identify performance bottlenecks and apply parallelisation using OpenMP and MPI to improve execution time. The results are then analysed in terms of speedup, efficiency, and scalability.

The platform used to execute all code in this work is Azure Labs. The VM has 12 CPUs. The GCC compiler was used for all executions, and the “-O0” flag was applied to all parallel implementations. Thread counts were varied to 1, 2, 4, 8, and 12 threads. Each code run was performed 3 times, and the average was reported. This was done to reduce noise from system activity. The dataset size was varied; the final run had 25000 data points.

The program opens a file and reads the number of data points, then allocates memory to the vector and matrix. Then it computes statistics and prints the results. The statistics being calculated include mean, variance, minimum absolute value, maximum absolute value and finally, an absolute eigenvalue of the matrix. The minimum absolute value and maximum absolute value calculations are the only parts that use OpenMP (`#pragma omp parallel for default(none) shared(n, x) private(val) reduction(max:maxabs)`). The OpenMP version in the original code only parallelises a small part of the total workload, so performance improvement was limited.

To start, a new directory was created (`hpcdb.coursework`). This is where the repo for this coursework was cloned (`git clone git@github.com:mkbane/mmu-hpc.git`). The baseline code ran extremely slowly. The eigenvalue computation is the primary bottleneck, accounting for the majority of execution time. The baseline OpenMP implementation showed almost no improvement as the thread count increased. For both 16K and 25K inputs, average runtime remained essentially unchanged across 1 to 12 threads. The lack of improvement with higher thread counts suggests that the algorithm is memory-bound and contains dependencies that prevent effective parallel execution. Parallel overheads such as thread creation, synchronisation, and communication reduce the effectiveness of parallelisation at higher levels.

A new copy of the code (`fullStat_driver-ASSESS.c`) was created so a more in-depth analysis could be performed while keeping the original version of the code untouched. The first improvement was to parallelise the mean and variance calculations. After these improvements, the results were collected in Table B. This improvement did not improve performance because only some vector operations were parallelised, whereas the dominant matrix-vector iterative eigenvalue computation was performed serially. As thread count increased, runtime performance declined slightly due to threading overhead. As more OpenMP was added to the loops, the results remained correct but the runtime still barely changed because the main bottleneck is the matrix-vector part in the eigenvalue iteration.

Finally, another copy of the code from the previous step was created. This time, the matrix-vector was targeted in parallelisation. At 25K data points, gains showed diminishing returns after thread 4. The noticeable diminishing returns are probably due to OpenMP overhead, memory bandwidth pressure, synchronisation overhead, and the fact that not every part of the program is equally expensive or equally parallel. Work distribution across threads/processes may not be perfectly balanced, which can also reduce performance gains.

It is clear from the results that increasing the number of threads does not significantly reduce execution time beyond a certain point. This is because the eigenvalue computation dominates the runtime and contains sequential dependencies that limit performance. This trend highlights that the parallel fraction of the code is limited, and increasing the number of threads leads to diminishing returns due to synchronisation overhead and memory contention. The eigenvalue computation remains the major cost, limiting overall scalability (OpenMP Architecture Review Board, 2021).

2a. Copilot Generated MPI Implementation

A fresh copy of the original MPI code (`fullStat_driver_mpi.c`) was downloaded to my PC using SCP. It was then opened on Visual Studio Code and then it was given to Copilot along with the prompt exactly as stated in

the appendix. The code produced by Copilot (`fullStat_driver_mpi_optimised.c`) was not compiled in this Visual Studio Code environment. It was downloaded and copied back into the Azure Labs Virtual Machine using SCP, and this is where it was compiled.

2b. Critique on Copilot MPI Code

The code from Copilot is a decent MPI solution, but the use of `MPI_Allreduce` with partially initialized arrays and incomplete normalisation across the ranks risks numerical consistency. MPI execution required oversubscription flags in the Azure Lab environment because Open MPI detected fewer available slots than the intended number of processes.

The MPI version generated by Copilot appeared to deliver dramatic performance gains for multiple processes, but correctness checks revealed that the eigenvalue output became 0 and the iteration count collapsed to 2 for all cases except the first one.

Although the implementation of MPI achieved significant runtime reductions, correctness was compromised, as shown by incorrect eigenvalues and premature convergence. Despite performance improvements, as the results are unreliable, it cannot be used for scientific computing.

This inconsistency in the results is from the incomplete initialisation of the arrays which also affected the use of `MPI_Allreduce`, as the data it operated on was incorrect. Also, the vectors were not completely normalised across all ranks and this affected the eigenvalue output.

MPI achieved strong speedup at lower process counts, but performance gains reduced as more processes were added. This is likely due to communication overhead and synchronisation between processes, especially during collective operations.

2c. Corrected MPI Implementation

To correct the code produced from Copilot, the whole `z` array was first normalised, not just the rows. This reduces redundancy as the version created by Copilot repeatedly normalises each row which leads to slowness of the code. Also, the whole `z` array was equated to zero before it was multiplied in the matrix. This was done to ensure numerical correctness across all process counts when compared to the reference code.

Performance improved substantially from 1 to 4 processes for both 16K and 25K inputs. Further gains were observed at 8 and 12 processes, although scaling became less efficient, indicating increasing communication overhead and diminishing returns (Open MPI, 2024).

On this VM, OpenMP performed better than MPI. For 16K data points:

- OpenMP 12 threads: 29.212179 s
- MPI 12 processes: 31.027770 s

For 25K data points:

- OpenMP 12 threads: 51.310230 s
- MPI 12 processes: 59.784866 s

The most likely reason for this is that shared-memory OpenMP had lower communication overhead, while MPI incurred additional broadcast and reduction costs. Additionally, oversubscription is likely to affect the MPI run with higher processes (8 and 12 processes).

Overall, OpenMP performed better than MPI for this task in the given environment. This is because OpenMP operates in shared memory, avoiding the communication overhead required by MPI. In contrast, MPI introduces additional costs due to data distribution and process synchronisation.

3. Scaling Estimate to 100 Cores

From the results for the best run for both OpenMP and MPI, it is clear that there are diminishing returns from 1 to 12 threads or processes. Therefore, a simple linear extrapolation would not be accurate to estimate 100 cores. Amdahl's law implies that the serial portion in the code limits the achievable speedup; therefore, a 100-core runtime cannot scale linearly like running it on 1 core.

Runtime on 100 cores was estimated using the results from the 25K final implementation of OpenMP and MPI. With the 12-thread run on OpenMP, the serial fraction was calculated to be 0.13. Using Amdahl's law ($T_p = \alpha T_1 + (1 - \alpha)T_1/p$) to estimate the scalability and substituting the serial fraction, the estimated runtime is about 36 seconds. For this same 12-thread run on OpenMP, the efficiency was calculated at 42.5%. If this same efficiency is used to estimate the 100-core run, the runtime would be approximately 6 seconds. The Amdahl's Law estimate (~ 36 seconds) is more realistic. This demonstrates that the program's serial portion fundamentally limits the achievable speedup.

For MPI, the 12-process run has an efficiency of 37.8%. This suggests that the 100-core runtime would be around 7 seconds. Real performance would probably be worse because of communication overhead. Using Amdahl's law to estimate the runtime, the serial fraction is calculated as 0.15, and the runtime for 100 cores is 43 seconds. This result is also more realistic than efficiency-based scaling. The results from the calculation can serve as upper bounds for the runtime estimate because they assume ideal scaling conditions. In practice, performance at 100 cores would likely be worse due to additional factors such as communication overhead (MPI), cache coherence traffic (OpenMP), and non-uniform memory access (NUMA effects).

For the implementation of OpenMP on 100 cores, a large shared-memory node is required. An infrastructure that has a very high memory bandwidth and RAM is required since all the nodes share one address space. Overhead communication in OpenMP will be low compared to MPI since all threads share the memory, but performance will still be affected by the degradation caused by memory contention and cache coherent traffic.

MPI is more natural to implement with 100 cores because it would run on a cluster of multiple nodes connected by a fast interconnection. No need for a single large shared-memory node like in OpenMP, but it would suffer from communication overhead, especially for global reductions; this would limit scaling at larger process counts.

Big Data

Amazon Product Review Analysis

This section analyses the Amazon electronics reviews dataset to investigate whether highly rated products show different review activity from lower-rated products in Q2 2014. The dataset contains user IDs, product IDs, ratings and timestamps, making it suitable for testing whether review behaviour differs across rating groups. Spark was used because the dataset contains millions of rows and is too large for simple desktop tools.

To make this testable, the hypotheses were refined as:

Research hypothesis: In Q2 2014, products with ratings of 3 or above show significantly higher review activity from products rated below 3.

Null hypothesis (H0): There is no significant difference in the mean number of reviews per product between products rated ≥ 3 and products rated < 3 in Q2 2014.

Alternative hypothesis (H1): There is a significant difference in the mean number of reviews per product between products rated ≥ 3 and products rated < 3 in Q2 2014.

The Process

The dataset was downloaded from Kaggle and transferred into the Azure Labs virtual machine using SCP. Since the raw dataset did not include column headers, appropriate headers were added based on the dataset description. The cleaned dataset was then stored in Hadoop (HDFS) for distributed processing.

Spark was used to analyse the dataset due to its ability to process large-scale data efficiently. The dataset contained over 7.8 million records, confirming that it qualifies as a big data problem, as the dataset is too large for tools like Excel and is better handled with distributed processing. Exploratory plots, including the Q2 2014 rating distribution and review-count comparison by rating group, are provided in Appendix B to support the statistical findings.

Initial exploration was carried out using Spark to understand key characteristics such as mean rating, distribution, and variability. Ratings of all records in the dataset varied from 1 to 5. The average rating across all products was approximately 4.01, indicating that the dataset is heavily skewed towards higher ratings (5). In fact, ratings of 4 and 5 accounted for over 74% of all records, suggesting a strong positive bias.

To focus on the hypothesis, the timestamp column was transformed into more useful features including year, month, and quarter, allowing filtering for Q2 2014. This subset contained 664,014 records.

The filtered dataset for Q2 2014 was divided into two groups based on rating: high ratings (≥ 3) and low ratings (< 3). In this period, the majority of records fell into the high ratings category, with 549,939 entries (approximately 83%), while the low ratings group contained 114,075 records (around 17%). This imbalance shows that higher-rated products dominate the dataset.

Table 1: Group Comparison

Metric	High Ratings	Low Ratings
Unique products	107,281	42,955
Average reviews per product	5.13	2.66
Maximum number of reviews	2468	903

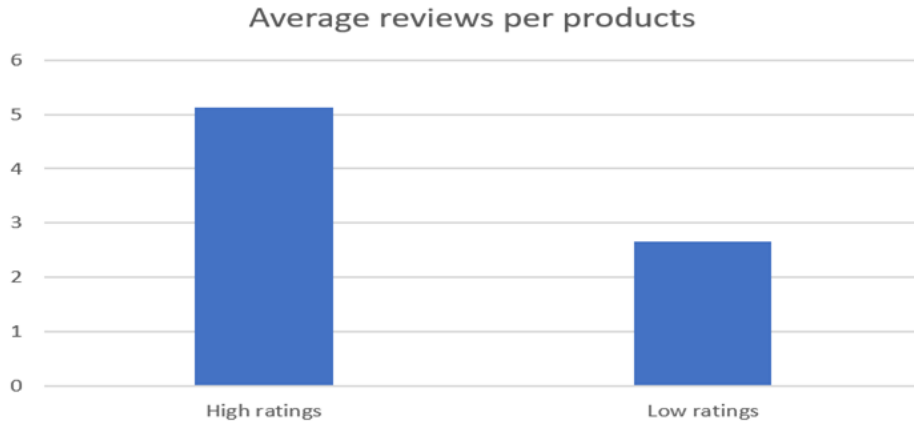


Figure 1: Comparison of average reviews per product between high-rated and low-rated

Products with ratings ≥ 3 consistently received more reviews per product, suggesting higher engagement. This indicates that better-rated products attract more user interaction, while poorly rated products receive less attention.

Statistical Testing

To validate these findings, a Welch’s t-test was conducted in Google Colab (due to better statistical tooling compared to Spark on the Azure Labs VM). A Welch’s t-test was used because the two rating groups had unequal sample sizes and likely unequal variance. The test returned $t \approx 30.09$ and $p \approx 0$, meaning the null hypothesis was rejected. This provides strong evidence that review activity differs between the two groups.

Conclusion

The null hypothesis is rejected. There is strong statistical evidence that, in Q2 2014, products with ratings ≥ 3 have significantly higher review activity than those with ratings < 3 .

The results clearly show a meaningful difference in user engagement between the two groups. While the analysis does not account for external factors such as product category, price, or the product brand, future work can include these variables or apply regression-based approaches to properly understand these relationships.

From a product strategy perspective, Amazon should monitor low-rated products with unusually low engagement, as they may reflect weak customer satisfaction or limited product visibility. Products with ratings ≥ 3 attracted more review activity, suggesting stronger customer engagement. Amazon could use this insight to prioritise recommendation visibility, seller quality checks, and review monitoring for poorly performing products (Fernandes et al., 2022).

Amazon data Pipeline (Step 3)

An end-to-end big data pipeline was implemented using Hadoop, Kafka, and Spark within the Azure Labs environment.

Historical review data was first stored in HDFS, providing distributed storage. A subset of this dataset was then converted into a JSON stream (reviews_stream) consisting of 1000 records to simulate real-time data.

Kafka was used for streaming ingestion (Apache Kafka, 2026). It distinguishes data producers from the consumers and makes sure that all incoming data can be continuously ingested without being impacted by the speed of downstream processing, making it more scalable and resilient. The use of Kafka enabled real-time ingestion, simulating a streaming environment, while Spark Structured Streaming provided near real-time processing. This reflects modern data architectures used in industry for handling continuous data flows.

The JSON data was pushed into a Kafka topic using a producer running in a separate terminal. There were some

minor challenges when connecting Kafka and Spark, but once the configuration was correct, it was processed as expected.

Spark Structured Streaming (in Scala) was then used to consume the Kafka stream. A schema was defined, and the incoming JSON data was parsed into a structured format. The data was then transformed and aggregated to compute product-level insights (Apache Spark, 2026).

Two outputs were created:

- Console output for real-time monitoring
- HDFS output (Parquet format) for persistent storage

The pipeline was validated by confirming:

- Kafka successfully received the stream
- Spark processed the stream in real time
- Output files were written to HDFS

The processed data was stored in parquet format because columnar storage format is efficient in dealing with big data. Also, it is easier to compress, allowing Spark to only read the required columns during analysis, thereby increasing speed.

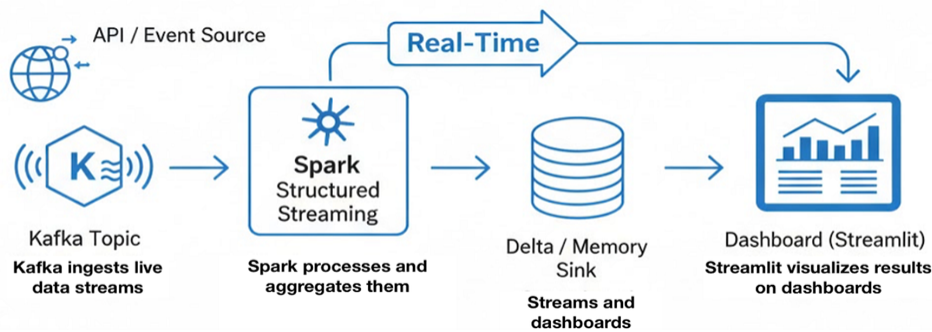


Figure 2: End-to-end big data pipeline integrating Kafka, Spark Structured Streaming, and HDFS

This demonstrates a complete integration of storage, streaming, and processing components in a single-node big data pipeline. Although this pipeline was implemented on a single-node, the architecture is scalable and reflects real-world big data systems with a single-node. Evidence of the pipeline was collected in the appendix.

References

1. Apache Hadoop (2023) *HDFS Architecture*. Available at: <https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html>
2. Apache Kafka (2026) *Apache Kafka Quickstart*. Available at: <https://kafka.apache.org/quickstart/>
3. Apache Spark (2026) *Structured Streaming Programming Guide*. Available at: <https://spark.apache.org/docs/latest/streaming/index.html>
4. OpenMP Architecture Review Board (2021) *OpenMP Application Programming Interface Version 5.2*. Available at: <https://www.openmp.org/specifications/>
5. Open MPI (2024) *Open MPI Documentation*. Available at: <https://docs.open-mpi.org/>
6. Fernandes, S., Panda, R., Venkatesh, V.G., Swar, B.N. and Shi, Y. (2022) ‘Measuring the impact of online reviews on consumer purchase decisions’, *Journal of Retailing and Consumer Services*, 68, 103066.
7. Statista (2024) *Online reviews influence on purchasing decisions*. Available at: <https://www.statista.com/>

A Appendix A: HPC Results

All experiments were executed three times, and the average values were used to improve reliability and reduce variability in measurements.

A.1 OpenMP Results

Results for 25K datapoint:

- Mean: -0.001371
- Std Dev: 5.784907
- Eigenvalue: 127425.006948
- Iterations: 46

Table 2: Baseline OpenMP performance on 16K datapoints

Threads	Run 1 (s)	Run 2 (s)	Run 3 (s)	Average (s)	Speedup	Efficiency
1	145.179162	148.725243	146.466953	146.790453	1.00	1.00
2	147.051726	146.534929	147.503608	147.030088	1.00	0.50
4	146.686164	146.926176	146.475526	146.695955	1.00	0.25
8	147.255592	146.447044	146.813652	146.838763	1.00	0.13
12	147.191181	147.588447	147.196413	147.325347	1.00	0.08

Table 3: Baseline OpenMP performance on 25K datapoints

Threads	Run 1 (s)	Run 2 (s)	Run 3 (s)	Average (s)	Speedup	Efficiency
1	267.573362	268.348606	265.638854	267.186941	1.00	1.00
2	267.406488	265.934565	269.638714	267.659922	1.00	0.50
4	269.284716	266.291929	266.507509	267.361385	1.00	0.25
8	269.176452	267.664200	267.749358	268.196670	1.00	0.13
12	267.622022	265.512263	269.939843	267.691376	1.00	0.08

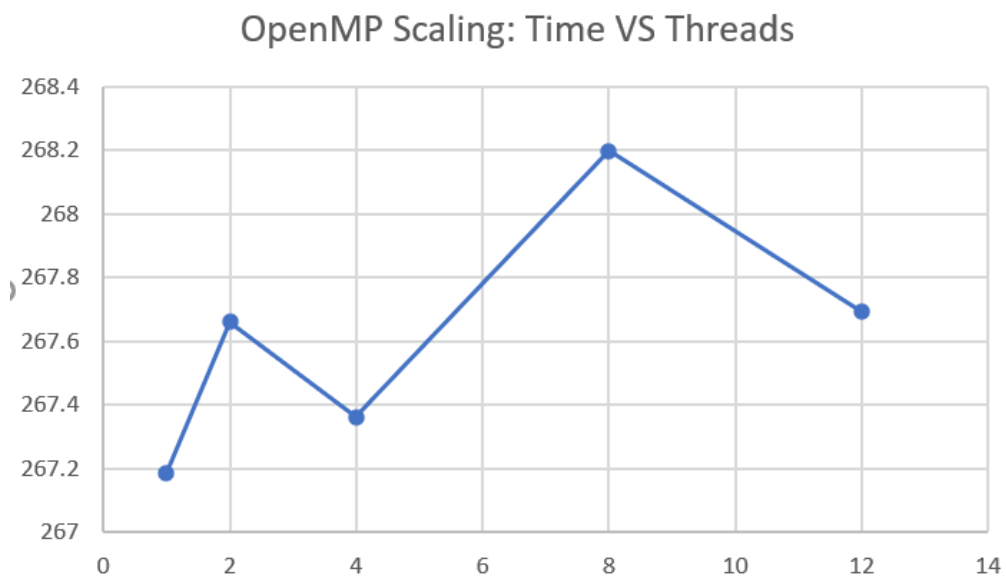


Figure 3: Time VS Threads for baseline run on 25K datapoints

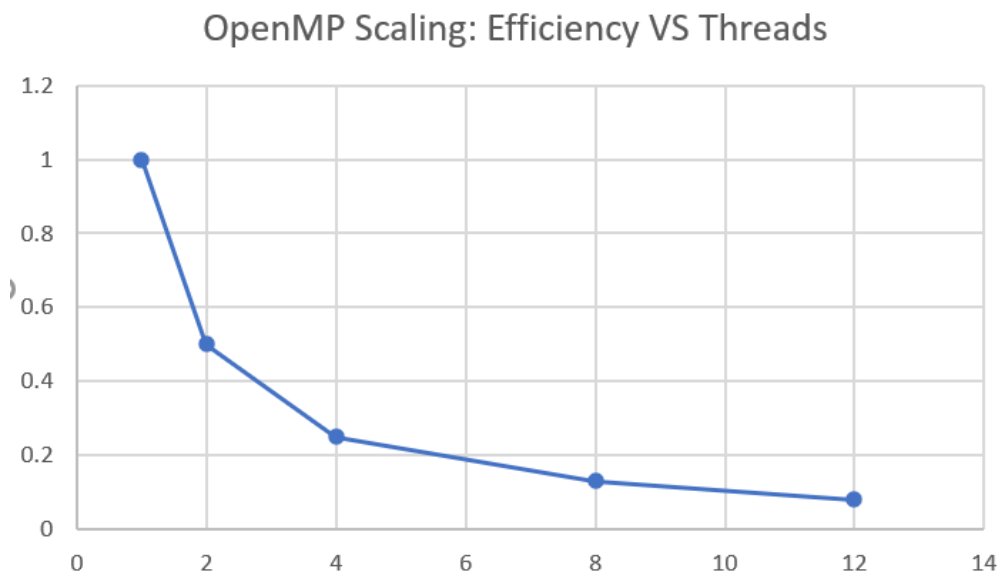


Figure 4: Efficiency VS Threads for baseline run on 25K datapoints

Table 4: Improved vector Parallelism 16K datapoints

Threads	Run 1 (s)	Run 2 (s)	Run 3 (s)	Average (s)	Speedup	Efficiency
1	144.779162	145.725243	146.466953	145.657119	1.00	1.00
2	146.057226	146.034929	146.203608	146.098588	1.00	0.50
4	145.986164	145.726176	145.475526	145.729289	1.00	0.25
8	145.255592	146.432044	144.813652	145.500429	1.00	0.13
12	146.191281	146.246447	146.124643	146.187457	1.00	0.08

Table 5: Improved vector Parallelism 25K datapoints

Threads	Run 1 (s)	Run 2 (s)	Run 3 (s)	Average (s)	Speedup	Efficiency
1	269.573362	269.348606	269.838854	269.586941	1.00	1.00
2	269.906488	269.934565	269.638714	269.826589	1.00	0.50
4	270.264716	270.291929	270.507569	270.354738	1.00	0.25
8	267.176452	266.664200	266.346358	266.729003	1.00	0.13
12	265.549022	265.578263	265.990265	265.705850	1.00	0.08

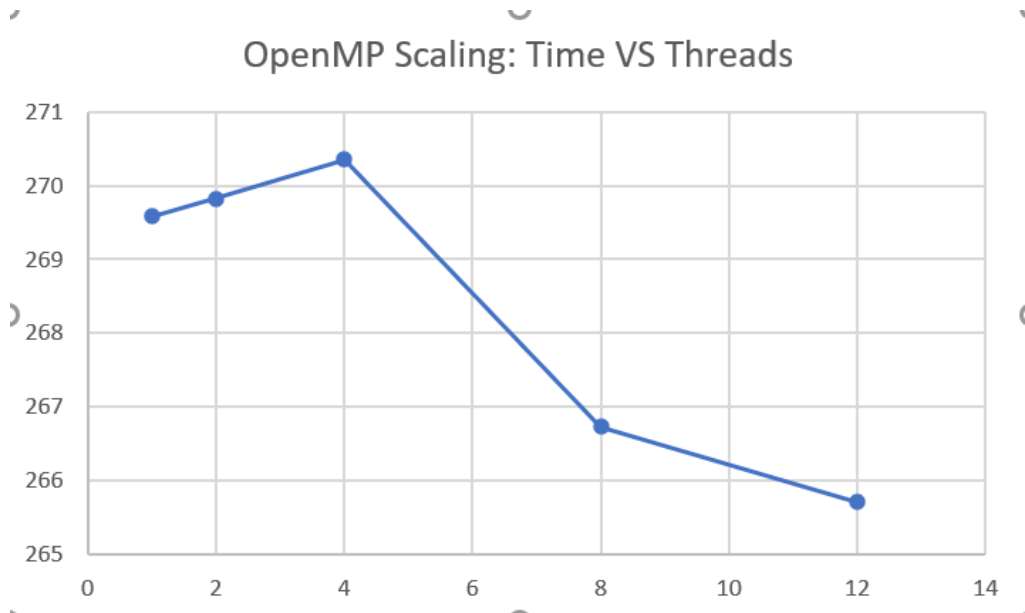


Figure 5: Time VS Threads for improved vector parallelism run on 25K datapoints

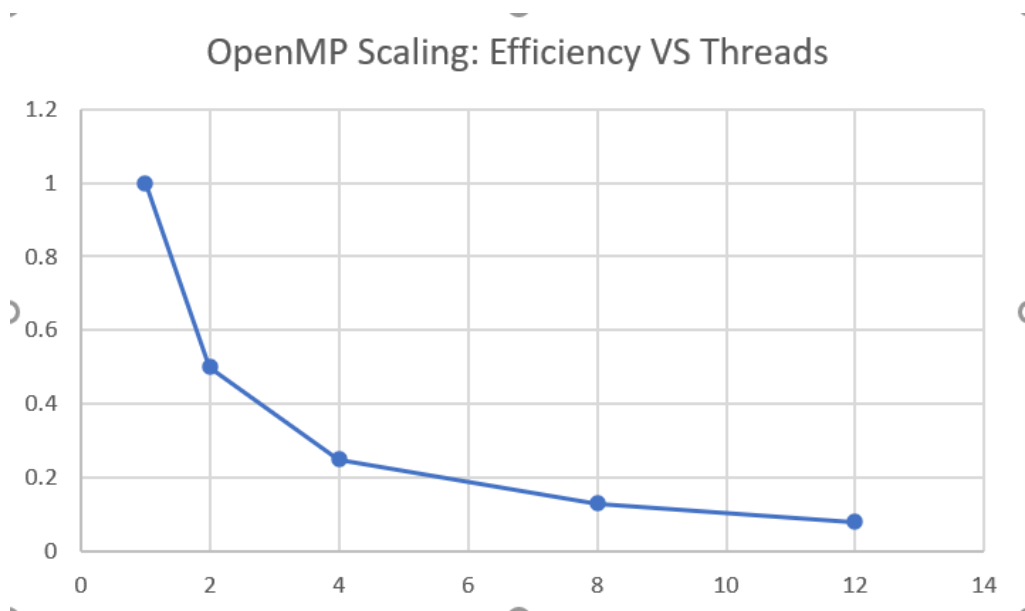


Figure 6: Efficiency VS Threads for improved vector parallelism run on 25K datapoints

Table 6: Full Parallelisation on 16K datapoints

Threads	Run 1 Total (s)	Run 2 Total (s)	Run 3 Total (s)	Average Total (s)	Speedup	Efficiency
1	142.628651	141.593848	146.915595	143.712698	1.00	1.00
2	81.816903	81.606698	79.308455	80.910685	1.78	0.89
4	45.211158	44.291754	43.265968	44.256293	3.25	0.81
8	31.947418	32.514446	32.204070	32.221978	4.46	0.56
12	29.037103	28.795116	29.804319	29.212179	4.92	0.41

Table 7: Full Parallelisation on 25K datapoints

Threads	Run 1 Total (s)	Run 2 Total (s)	Run 3 Total (s)	Average Total (s)	Speedup	Efficiency
1	258.121454	258.004660	269.163532	261.763215	1.00	1.00
2	131.677775	131.365944	136.140959	133.061559	1.97	0.99
4	68.174321	67.816658	69.723394	68.571458	3.82	0.96
8	51.127948	51.058047	52.822027	51.669341	5.07	0.63
12	50.856688	50.190851	52.883152	51.310230	5.10	0.43

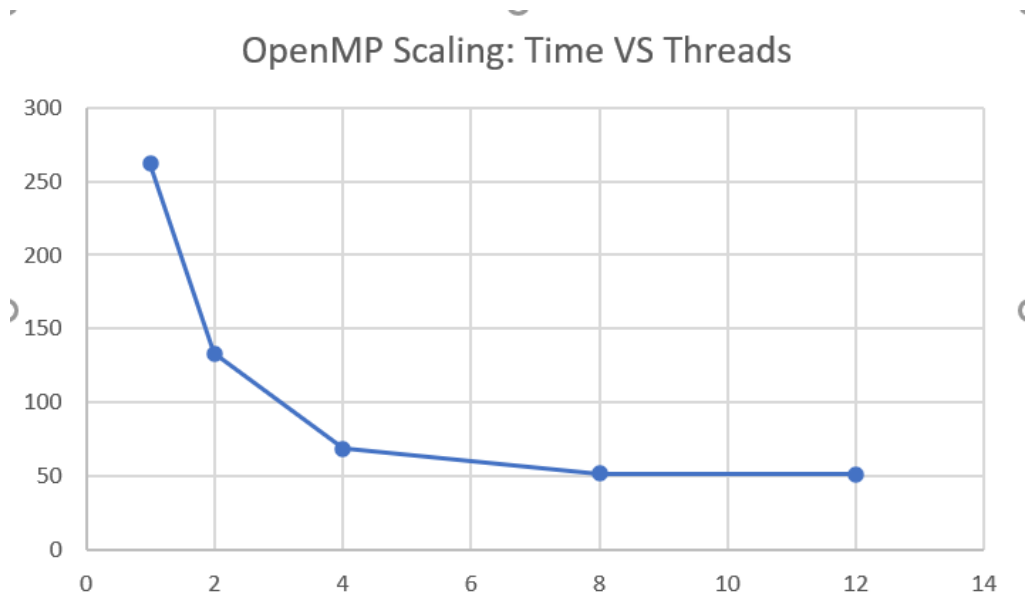


Figure 7: Time VS Threads for Full parallelism run on 25K datapoints

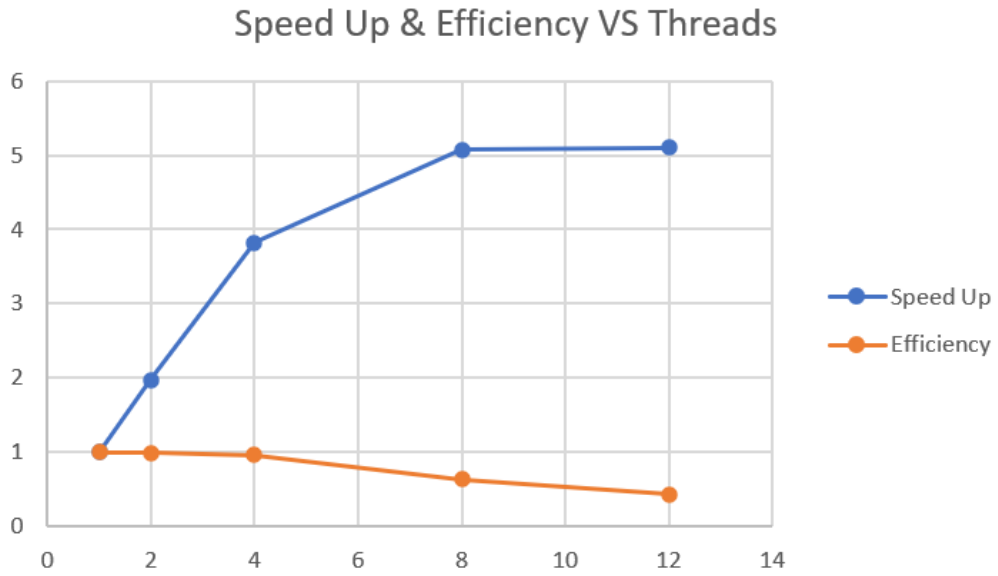


Figure 8: Speed Up & Efficiency VS Threads for full parallelism on 25K datapoints

A.2 MPI Results

Correct reference results for 16K:

- Mean = 0.056773
- Std Dev = 5.792664
- Eigenvalue = 81572.218349
- Iterations = 47

Table 8: Co-pilot code for 16K datapoints

Processes	Run 1	Run 2	Run 3	Average	Mean	Std Dev	Eigenvalue	Correct?
1	145.562118	143.318886	145.569832	144.816945	0.056773	5.792664	81572.218349	Yes
2	3.425596	3.418003	3.365235	3.402945	0.056773	5.792664	0.000000	No
4	1.801896	1.767672	1.765916	1.778495	0.056773	5.792664	0.000000	No
8	1.523106	1.729750	1.534953	1.595936	0.056773	5.792664	0.000000	No
12	1.265929	1.515573	1.197283	1.326262	0.056773	5.792664	0.000000	No

Correct reference results for 25K:

- Mean = -0.001371
- Std Dev = 5.784907
- Eigenvalue = 127425.006948
- Iterations = 46

Table 9: Co-pilot code for 25K datapoints

Processes	Run 1	Run 2	Run 3	Average	Mean	Std Dev	Eigenvalue	Correct?
1	270.798149	270.380593	270.168214	270.448985	-0.001371	5.784907	127425.006948	Yes
2	5.905603	5.937340	5.993619	5.945521	-0.001371	5.784907	0.000000	No
4	3.136515	3.196188	3.211384	3.181362	-0.001371	5.784907	0.000000	No
8	2.702768	2.740019	2.710425	2.717737	-0.001371	5.784907	0.000000	No
12	2.660480	2.611915	2.673414	2.648603	-0.001371	5.784907	0.000000	No

Table 10: MPI results for 16K datapoints (Edited version)

Processes	Run 1 (s)	Run 2 (s)	Run 3 (s)	Average (s)	Speedup	Efficiency
1	138.461669	138.970164	138.988886	138.806906	1.00	1.00
2	77.203052	74.686498	79.697081	77.195544	1.80	0.90
4	44.421293	43.203234	45.606507	44.410345	3.13	0.78
8	36.827834	38.693817	34.697659	36.739770	3.78	0.47
12	30.904501	32.287881	29.890927	31.027770	4.47	0.37

Table 11: MPI results for 25K datapoints (Edited version)

Processes	Run 1 (s)	Run 2 (s)	Run 3 (s)	Average (s)	Speedup	Efficiency
1	276.134218	268.346468	267.951527	270.810738	1.00	1.00
2	140.423367	140.532068	140.716253	140.557229	1.93	0.97
4	73.591243	75.789479	73.654540	74.345087	3.64	0.91
8	61.639051	61.540714	61.674211	61.617992	4.40	0.55
12	59.133450	60.145675	60.075474	59.784866	4.53	0.38

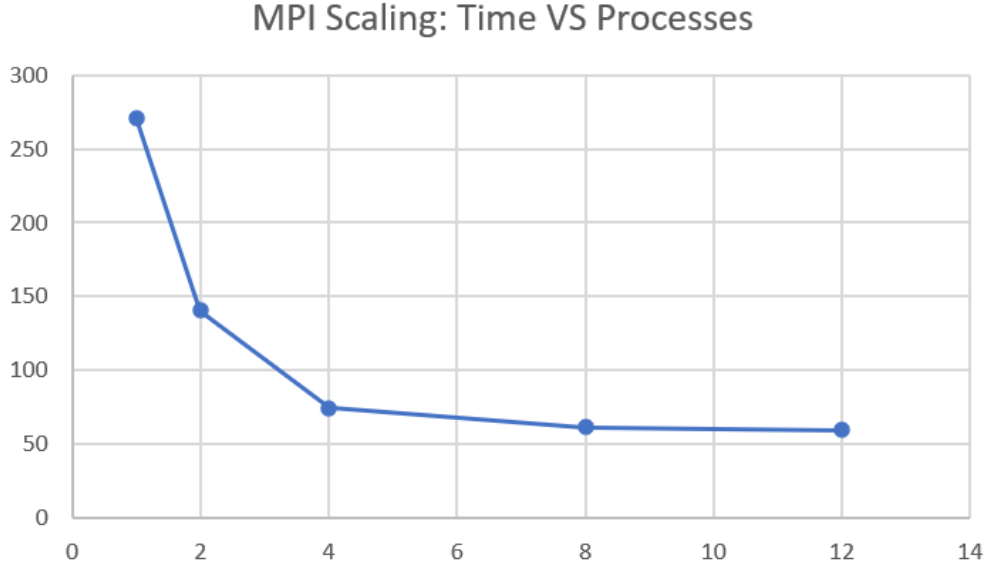


Figure 9: Time VS Processes for final MPI run on 25K datapoints

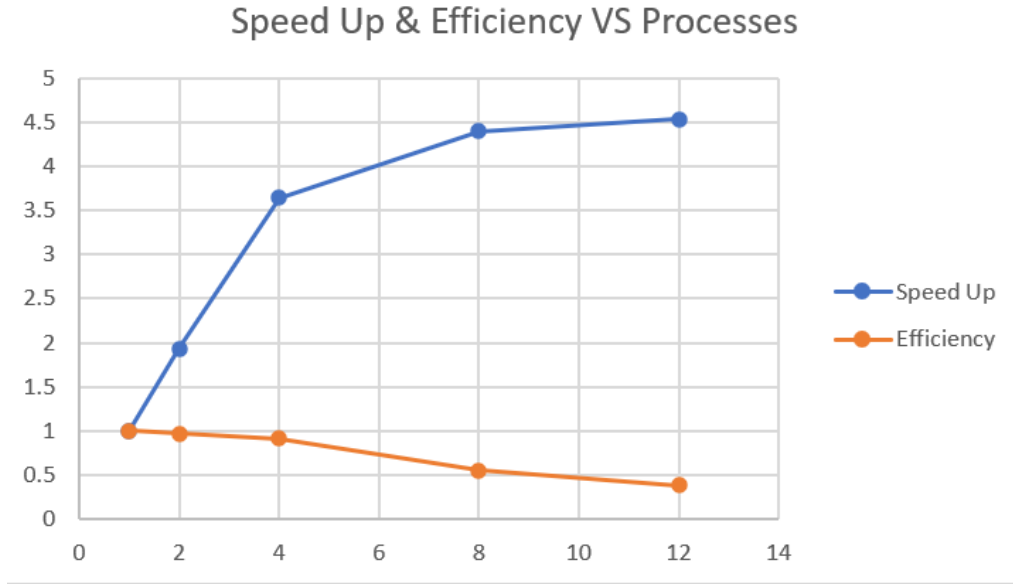


Figure 10: Speed Up & Efficiency VS Processes for final MPI run on 25K datapoints

A.3 Formulas

$$\text{Speedup} = T_1 / T_p$$

$$\text{Efficiency} = \text{Speedup} / \text{Number of threads or Processes}$$

Amdahl's Law:

$$T_p = \alpha T_1 + \frac{(1 - \alpha) T_1}{p}$$

A.4 Commands Used

- `gcc -O0 -fopenmp fullStat_driver-ASSESS.c autogenRoutines.c -lm -o fullstat`

The `-fopenmp` flag enables OpenMP parallelism, while `-O0` ensures no compiler optimisations interfere

with performance measurement.

- gcc -O0 -fopenmp fullStat_driver_q1.c autogenRoutines.c -lm -o fullstat_q1
- export OMP_NUM_THREADS=1
- ./fullstat 1K.txt
- Lscpu

Lscpu was used to check the number of CPUs on the VM.

- mpicc -O0 fullStat_driver_mpi_optimized.c autogenRoutines.c -lm -o fullstat_mpi_copilot
- mpicc -O0 fullStat_driver_mpi_optimised_edited.c autogenRoutines.c -lm -o fullstat_mpi_fixed
- mpirun --allow-run-as-root -np 4 ./fullstat_mpi_fixed 25K.txt
- mpirun --allow-run-as-root --oversubscribe -np 8 ./fullstat_mpi_fixed 25K.txt

```
Eigen value of matrix: 127425.006948 (46 iterations)
Completed (read+stats). [276.671385 seconds]
vm_admin@labY70131:~/hpcbd_coursework/mmu-hpc/2025-2026$ export OMP_NUM_THREADS=12
vm_admin@labY70131:~/hpcbd_coursework/mmu-hpc/2025-2026$ ./fullstat_q1 25K.txt
we have 25000 data points to read
There are allegedly 25000 data points to read
range 20.000000: from -10.000000 to 10.000000
normalise: 9.31323e-09
RAND_MAX: 2147483647
25000 data points successfully read [9.801920 seconds]
x[0]=6.803754
Time for vector stats: 0.000512 seconds
calculating eigenvalue to tolerance 0.010000
Time for eigen stats: 265.803768 seconds
Total stats wallclock time [265.804328 seconds]
Stats table
Min abs val      Max abs val      Mean              Std Dev
0.000597         9.999855        -0.001371         5.784907
Eigen value of matrix: 127425.006948 (46 iterations)
Completed (read+stats). [275.606379 seconds]
```

Figure 11: Execution of 12 threads run on 25K datapoints for OpenMP

A.5 Numerical Accuracy statement

Execution time was measured using `omp_get_wtime()` as implemented in the code. Each configuration was run three times and averaged for improved accuracy.

Numerical accuracy was checked by comparing mean, standard deviation, eigenvalue and iteration count across baseline, OpenMP and MPI runs. The final OpenMP and corrected MPI implementations preserved the expected outputs for both 16K and 25K datasets, while the unedited Copilot MPI version produced incorrect eigenvalues for more than one process.

The second question was performed on 19th March, 2026 on Visual Studio Code for Copilot editing and on Azure Labs VM.

A.6 Copilot

Copilot version: VS code extension: GitHub Copilot chat, version 0.40.1

Prompt- User: Convert this C program into an MPI parallel version using standard MPI (mpicc). Modify the driver file. Keep autogenRoutines.c separate. Distribute the computation across processes. Use MPI collective communication where appropriate and keep the output as similar as possible. Only parallelise the computationally intensive parts, keep file reading on rank 0.

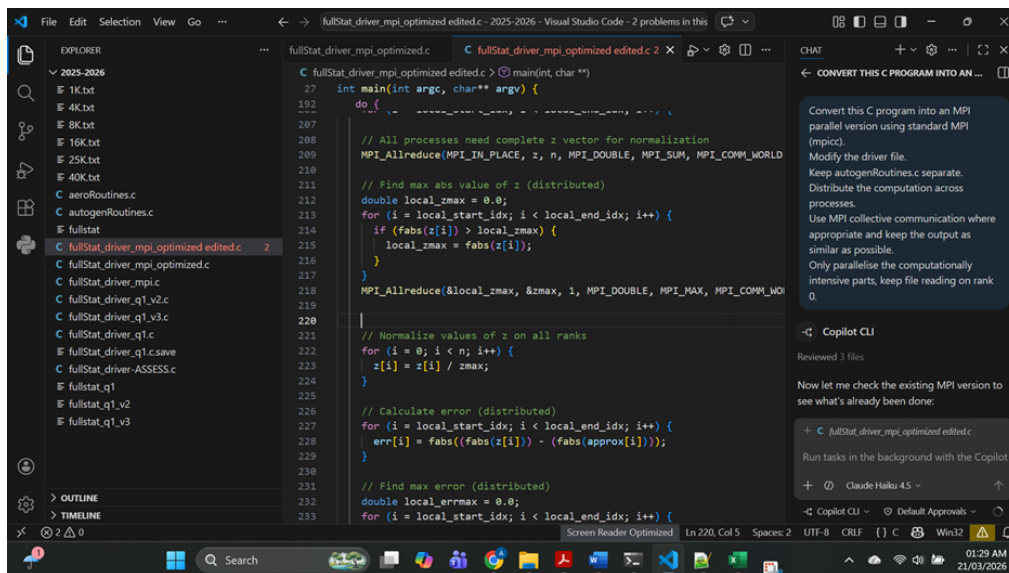


Figure 12: Copilot interface on VS Code

Read `fullStat_driver_mpi.c`

Perfect! Now I understand the structure. Let me create an MPI parallel version of the driver. Here's my approach:

Key parallelization strategy:

1. **File reading (rank 0 only)** - Read data and broadcast to all processes
2. **Vector statistics** - Distribute vector calculations across processes using MPI collective operations
3. **Matrix-vector multiplication** - Distribute rows of matrix across processes
4. **Reductions** - Use MPI_Allreduce for min/max/sum operations

Let me create the MPI version:

Updated `fullStat_driver_mpi_optimized.c`

Figure 13: Copilot response and creation of the new code

A.7 Amdahl's Law Calculations

OpenMP scaling calculation with Amdahl's law

$$T_1 = 261.763215, \quad T_{12} = 51.310230$$

$$S_{12} = \frac{T_1}{T_{12}} = \frac{261.763215}{51.310230} \approx 5.10$$

$$S_p = \frac{1}{\alpha + \frac{1-\alpha}{p}}$$

$$5.10 = \frac{1}{\alpha + \frac{1-\alpha}{12}}$$

$$\alpha \approx 0.13$$

$$T_{100} = \alpha T_1 + \frac{(1-\alpha)T_1}{100}$$

$$T_{100} = (0.13 \times 261.763215) + \frac{(1-0.13) \times 261.763215}{100}$$

$$T_{100} \approx 36 \text{ seconds}$$

MPI scaling calculation with Amdahl's law

$$T_1 = 270.810738, \quad T_{12} = 59.784866$$

$$S_{12} = \frac{T_1}{T_{12}} = \frac{270.810738}{59.784866} \approx 4.53$$

$$4.53 = \frac{1}{\alpha + \frac{1-\alpha}{12}}$$

$$\alpha \approx 0.15$$

$$T_{100} = \alpha T_1 + \frac{(1-\alpha)T_1}{100}$$

$$T_{100} = (0.15 \times 270.810738) + \frac{(1-0.15) \times 270.810738}{100}$$

$$T_{100} \approx 43 \text{ seconds}$$

B Appendix B: Big Data and Pipeline

The appendix includes:

- Google Colab notebook (Welch t-test)
- All code used in Azure Labs (saved separately in notepad)
- Screenshots showing:

- Kafka topic creation and producer
- Spark streaming setup (Scala)
- JSON stream creation
- Aggregation process
- Streaming output
- HDFS Parquet output

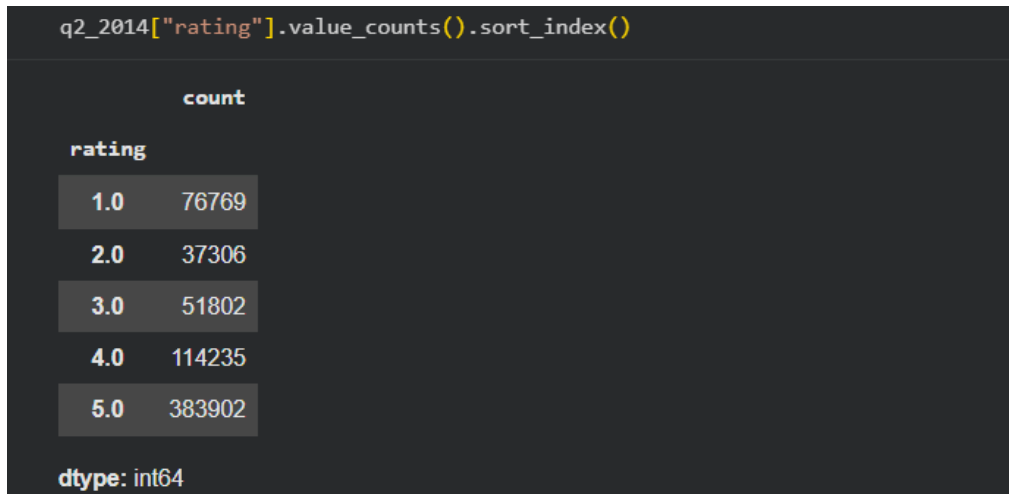


Figure 14: Q2 2014 size

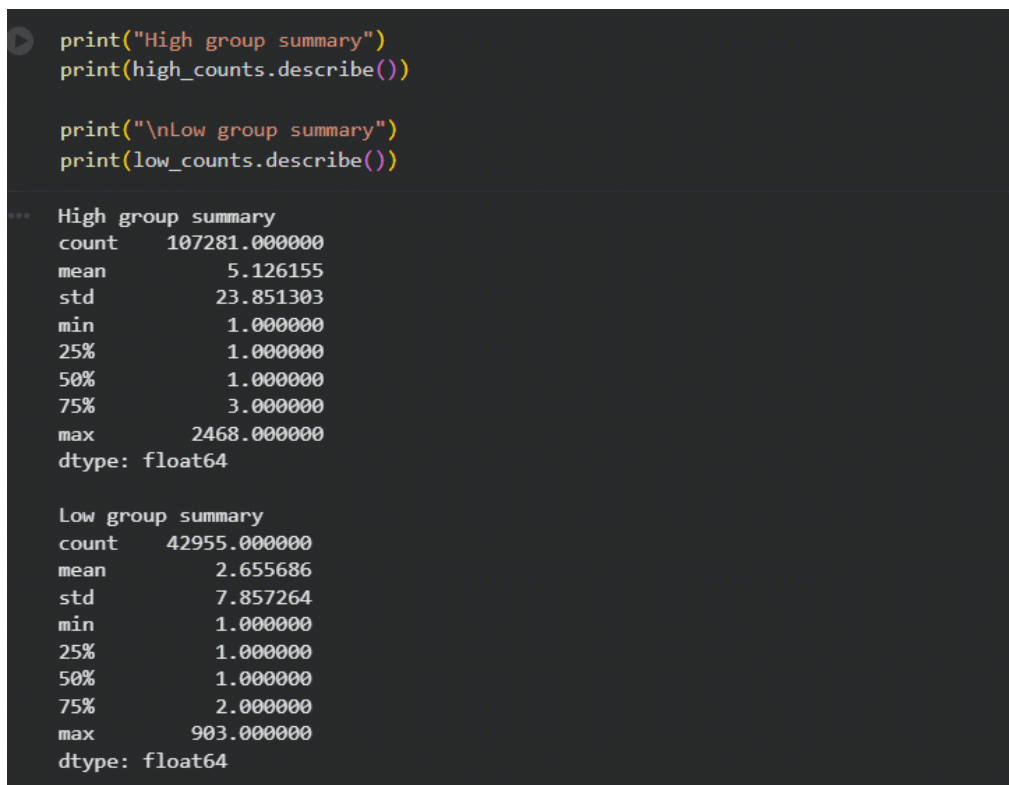


Figure 15: Summary statistics for both groups

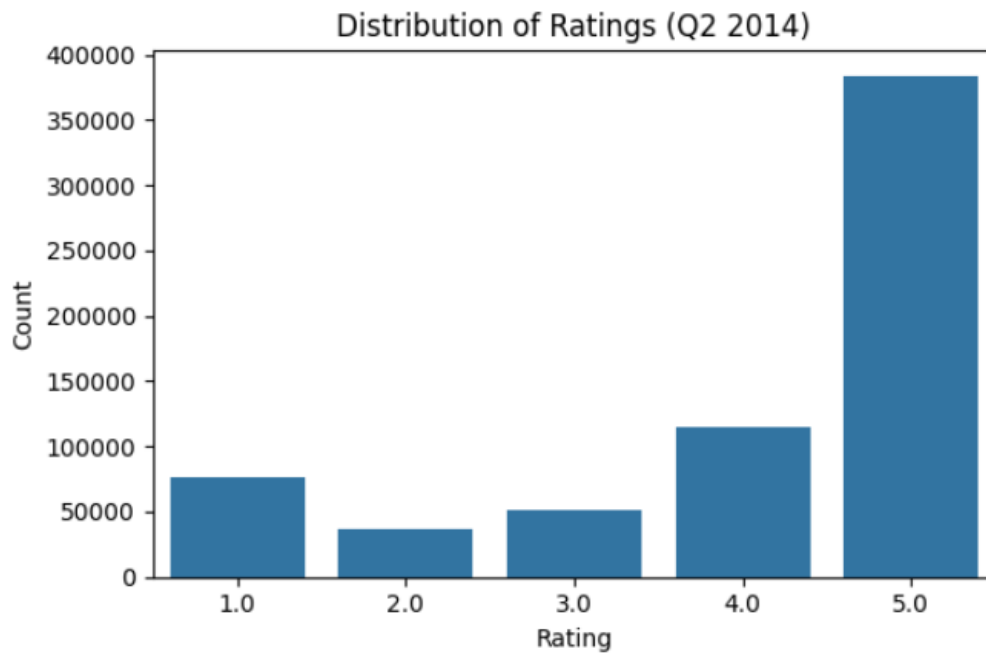


Figure 16: Distribution of Ratings (EDA)

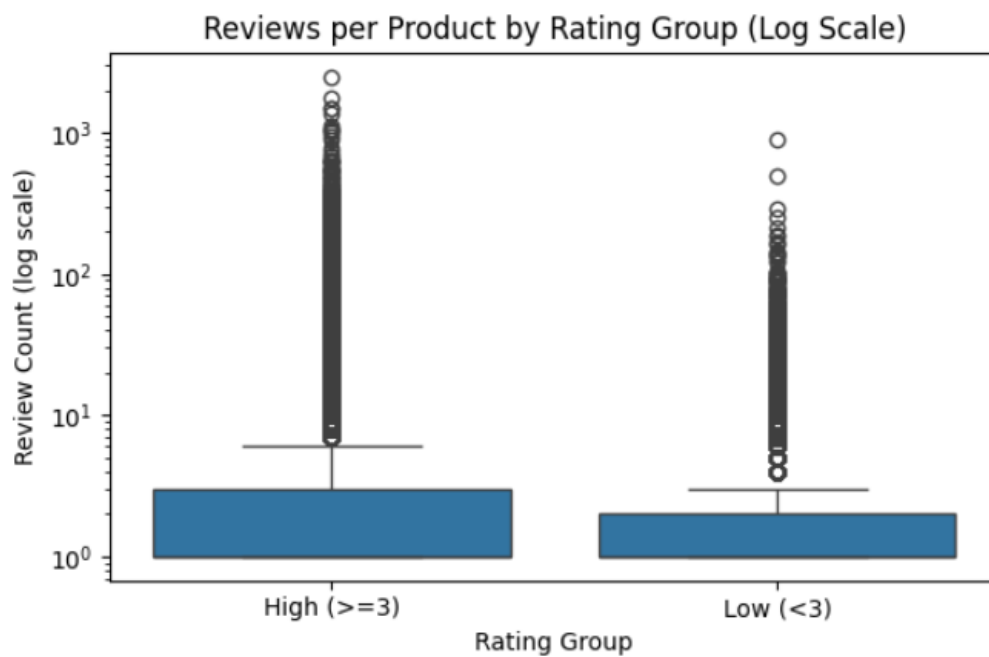


Figure 17: Reviews per Product based on the rating groups (EDA)

```
t_stat, p_value = stats.ttest_ind(high_counts, low_counts, equal_var=False)

print("Welch t-statistic:", t_stat)
print("p-value:", p_value)

Welch t-statistic: 30.091901799832016
p-value: 2.5006391891054197e-198
```

Figure 18: Welch t-test results

B.1 Pipeline (Step 3)

```
vm_admin@labY7013I:~$ jps
4834 SecondaryNameNode
4395 NameNode
34859 Jps
10173 Kafka
4558 DataNode
15407 SparkSubmit
```

Figure 19: Kafka, Spark, and HDFS working

```
vm_admin@labY7013I:~/kafka_2.13-3.7.0$ jps
10289 Jps
4834 SecondaryNameNode
4395 NameNode
10173 Kafka
4558 DataNode
vm_admin@labY7013I:~/kafka_2.13-3.7.0$ bin/kafka-topics.sh --create \
> --topic reviews \
> --bootstrap-server localhost:9092 \
> --partitions 1 \
> --replication-factor 1
Created topic reviews.
vm_admin@labY7013I:~/kafka_2.13-3.7.0$
vm_admin@labY7013I:~/kafka_2.13-3.7.0$ bin/kafka-topics.sh --list --bootstrap-server localhost:9092
reviews
vm_admin@labY7013I:~/kafka_2.13-3.7.0$
vm_admin@labY7013I:~/kafka_2.13-3.7.0$
```

Figure 20: Creation of Kafka topic

```
vm_admin@labY7013I:~/kafka_2.13-3.7.0$ cd ~/bigdata_coursework
vm_admin@labY7013I:~/bigdata_coursework$
vm_admin@labY7013I:~/bigdata_coursework$ python3 - <<'PY'
> import csv, json, itertools
> with open('ratings.csv', newline='') as f, open('reviews_stream.json', 'w') as out:
>     reader = csv.DictReader(f)
>     for row in itertools.islice(reader, 1000):
>         out.write(json.dumps(row) + '\n')
> PY
vm_admin@labY7013I:~/bigdata_coursework$
vm_admin@labY7013I:~/bigdata_coursework$ head -3 ~/bigdata_coursework/reviews_stream.json
{"user_id": "AKM1MP6P00YPR", "item_id": "0132793040", "rating": "5.0", "timestamp": "1365811200"}
{"user_id": "A2CX7LU0HB2NDG", "item_id": "0321732944", "rating": "5.0", "timestamp": "1341100800"}
{"user_id": "A2NWSAGRHC8N5", "item_id": "0439886341", "rating": "1.0", "timestamp": "1367193600"}
vm_admin@labY7013I:~/bigdata_coursework$ wc -l ~/bigdata_coursework/reviews_stream.json
1000 /home/vm_admin/bigdata_coursework/reviews_stream.json
vm_admin@labY7013I:~/bigdata_coursework$
vm_admin@labY7013I:~/bigdata_coursework$
```

Figure 21: Creating the JSON stream

```
scala> val reviewSchema = new StructType().add("user_id", StringType).add("item_id", StringType).add("rating", StringType).add("timestamp", StringType)
reviewSchema: org.apache.spark.sql.types.StructType = StructType(StructField(user_id,StringType,true),StructField(item_id,StringType,true),StructField(rating,StringType,true),StructField(timestamp,StringType,true))
```

Figure 22: Defining the schema of data to be streamed

```
scala> val parsed = rawKafka.selectExpr("CAST(value AS STRING) as json_str").select(from_json(col("json_str"), reviewSchema).as("data")).select("data.*").withColumn("rating", col("rating").cast("double")).withColumn("event_time", timestamp(from_unixtime(col("timestamp")).cast("long"))))
parsed: org.apache.spark.sql.DataFrame = [user_id: string, item_id: string ... 3 more fields]

scala>

scala> val agg = parsed.groupBy("item_id").agg(count("*").as("review_count"), round(avg("rating"), 2).as("avg_rating"))
agg: org.apache.spark.sql.DataFrame = [item_id: string, review_count: bigint ... 1 more field]

scala>

scala>

scala> val consoleQuery = agg.writeStream.outputMode("complete").format("console").option("truncate", "false").start()
26/03/25 13:29:39 WARN ResolveWriteToStream: Temporary checkpoint location created which is deleted normally when the query didn't fail: /tmp/temporary-b256ec22-d9cc-43a6-bfa7-6dfc2482934d. If it's required to delete it under any circumstances, please set spark.sql.streaming.forceDeleteTempCheckpointLocation to true. Important to know deleting temp checkpoint folder is best effort.
26/03/25 13:29:39 WARN ResolveWriteToStream: spark.sql.adaptive.enabled is not supported in streaming DataFrames/Datasets and will be disabled.
consoleQuery: org.apache.spark.sql.streaming.StreamingQuery = org.apache.spark.sql.execution.streaming.StreamingQueryWrapper@16b0a770
```

Figure 23: Parsing and aggregating the data

```
scala> val rawKafka = spark.readStream.format("kafka").option("kafka.bootstrap.servers", "localhost:9092").option("subscribe", "reviews").option("startingOffsets", "earliest").load()
rawKafka: org.apache.spark.sql.DataFrame = [key: binary, value: binary ... 5 more fields]
```

Figure 24: Starting stream

```
scala> 26/03/25 13:29:39 WARN AdminClientConfig: These configurations '[key.deserializer, value.deserializer, enable.auto.commit, max.poll.records, auto.offset.reset]' were supplied but are not used yet.
-----
Batch: 0
-----
+-----+-----+-----+
|item_id|review_count|avg_rating|
+-----+-----+-----+
|0528881469|27|2.85|
|0594450705|1|5.0|
|0777700018|2|5.0|
|0594033896|5|4.4|
|0970407998|5|3.2|
|0594450209|2|5.0|
|0594287995|1|5.0|
|0594296420|6|4.67|
|0594549558|1|5.0|
|0594033926|15|4.53|
|0439886341|3|1.67|
|0594017580|1|3.0|
|0594481813|31|4.23|
|0972683275|817|4.47|
|089933623X|2|2.5|
|0558835155|1|3.0|
|0594482127|1|4.0|
|0594202442|1|4.0|
|0132793040|1|5.0|
|0594033934|2|5.0|
+-----+-----+-----+
only showing top 20 rows
```

Figure 25: Top 20 rows of data in JSON file being streamed

```
vm_admin@labY70131:~/kafka_2.13-3.7.0$
vm_admin@labY70131:~/kafka_2.13-3.7.0$ cd ~/kafka_2.13-3.7.0
vm_admin@labY70131:~/kafka_2.13-3.7.0$
vm_admin@labY70131:~/kafka_2.13-3.7.0$ bin/kafka-console-producer.sh --bootstrap-server localhost:9092 --topic reviews < ~/bigdata_coursework/reviews_stream.json
vm_admin@labY70131:~/kafka_2.13-3.7.0$
```

Figure 26: Data was actively streamed into Kafka

```
scala> val rawToHdfs = parsed.writeStream.format("parquet").option("path", "hdfs:///user/vm_admin/output/reviews_raw").option("checkpointLocation", "hdfs:///user/vm_admin/checkpoints/reviews_raw").outputMode("append").start()
org.apache.hadoop.HadoopIllegalArgumentException: Uri without authority: hdfs://user/vm_admin/output/reviews_raw/_spark_metadata
    at org.apache.hadoop.fs.AbstractFileSystem.getUri(AbstractFileSystem.java:329)
    at org.apache.hadoop.fs.AbstractFileSystem.<init>(AbstractFileSystem.java:282)
    at org.apache.hadoop.fs.Hdfs.<init>(Hdfs.java:80)
    at java.base/jdk.internal.reflect.NativeConstructorAccessorImpl.newInstance0(Native Method)
```

Figure 27: The pipeline was validated by confirming that Kafka successfully received streamed data, Spark processed it in real time, and the output was written to HDFS in Parquet format.


```

vm_admin@labY7013I:~$ hdfs dfs -ls /user/vm_admin/output/reviews_raw
Found 4 items
drwxr-xr-x  - vm_admin supergroup          0 2026-03-25 13:51 /user/vm_admin/output/reviews_raw/_spark_metadata
-rw-r--r--  1 vm_admin supergroup        23217 2026-03-25 13:51 /user/vm_admin/output/reviews_raw/part-00000-52c0ec24-2f6
2-4613-b34e-87e6a6babf3f-c000.snappy.parquet
-rw-r--r--  1 vm_admin supergroup         6137 2026-03-25 13:51 /user/vm_admin/output/reviews_raw/part-00000-5d579b2c-d07
0-4c2c-9d9a-fc506735db8e-c000.snappy.parquet
-rw-r--r--  1 vm_admin supergroup        26906 2026-03-25 13:50 /user/vm_admin/output/reviews_raw/part-00000-654a9a20-886
3-4f75-8797-86e7c22f973c-c000.snappy.parquet

```

Figure 28: Streaming data successfully persisted to HDFS

C Appendix C: Source Code Access

All the project code, scripts, logs, and output data have been uploaded to a private GitHub repository. Access has been granted to the course tutors, m.bane@mmu.ac.uk and S.Ajao@mmu.ac.uk

Repository URL: [GitHub Repository](#)